

Nombre(s): <u>Héctor David Martínez Huamán</u>	
Apellidos: <u>Martínez Huamán</u>	
Código: <u>21200096</u> Fecha: <u>20/06/26</u>	

Puntaje Total: 20 puntos

Duración: 2 horas + 10 min para entrega

Modalidad: Preguntas Abiertas y Desarrollo de Código.

Instrucciones: Resuelva el examen de forma individual y a mano. Al finalizar, digitalice todas las páginas en un único archivo PDF denominado `prueba1.pdf` y súbalo al repositorio: github.com/illarek-lab/mobile_pruebas

Dentro del repositorio cree un branch `C<codigo_estudiante>` con un directorio con el formato: `C<codigo_estudiante>` y coloque en su interior el archivo: `prueba1.pdf` y realice su PR. **Ejemplo:** `C20251234/prueba1.pdf`

Valor: 2 pts c/u

1. El Patrón Repositorio como SSOT

Explique por qué el Repositorio se considera la "Única Fuente de Verdad" (SSOT) y una "fachada" en la arquitectura. Detalle cómo coordina los datos entre la base de datos local (Room) y la API remota (Retrofit) para que la UI no sepa de dónde vienen los datos.

2. Corrutinas y Structured Concurrency

Defina qué es el `viewModelScope` y por qué es fundamental para la gestión de memoria. Explique qué sucede con una petición de red iniciada en este scope si el usuario decide cerrar la pantalla antes de recibir la respuesta.

3. Flujos Reactivos (Cold vs. Hot)

Diferencie técnicamente un `Flow` (frío) de un `StateFlow` (caliente). ¿Por qué para el manejo de estados de la interfaz de usuario se prefiere `StateFlow` sobre un `Flow` convencional?

4. Inyección de Dependencias (DI) Manual

Describa el mecanismo de "DI Manual" utilizado en los laboratorios mediante la clase `Application`. Explique para qué sirve la palabra reservada `by lazy` en la inicialización de los objetos de infraestructura (como la BD o el Repositorio).

Instrucción: Escriba el término técnico exacto que completa la oración basándose en el texto de las sesiones.

1. El flujo de datos que emite automáticamente cada vez que una tabla de Room cambia se implementa usando el tipo Flow.

I. 1. El repositorio centraliza las consultas a las fuentes de datos, dentro del patrón MVVM actúa como capa de intercambio entre la lógica y los datos. Lo coordina.

2. El viewmodelscope es la forma de implementar un Scope asociado a una viewmodel. Es fundamental para gestionar peticiones en red ya que una vez el usuario salga de la aplicación se cierran las peticiones recibidas. De esta forma se puede gestionar eficientemente la memoria.

3. El Flow permite la implementación de corutinas usando la configuración del manejo de procesos de forma concurrente. El stateflow maneja los estados dentro de una app manejando siempre el último, se prefiere usar stateflow ya que nos permite una recomposición reactiva de la UI.

4.

2. El operador AsyncTask permite ejecutar tareas pesadas fuera del Main Thread, específicamente optimizado para tareas de CPU como el parseo masivo de datos.
3. Para que un servicio en primer plano de ubicación sea válido en Android 10+, debe declararse en el manifiesto con el tipo Service.
4. La función onStart() de la clase Application es la que garantiza que los componentes compartan la misma instancia del repositorio mediante un "cast".
5. El componente que permite transformar una API basada en callbacks antiguos en una función suspendible moderna se llama Coroutines.
6. El archivo de metadatos que describe la estructura de la app al sistema operativo se llama exactamente androidManifest.xml.
7. En la arquitectura AOSP, la capa que actúa como puente estándar entre el framework y los controladores se denomina HAL.
8. El operador de Flow que permite combinar múltiples fuentes (como GPS y Sensores) para emitir un estado unificado es combineLatest3().

13. El operador de Flow que permite combinar múltiples fuentes (como GPS y Sensores) para emitir un estado unificado es combineLatest3().

Instrucción: Escriba el código en Kotlin siguiendo las convenciones de PascalCase para clases y camelCase para variables.

1. (2 pts) Repositorio y DI

Escriba el código de una clase `UsuarioRepository` que reciba por constructor un DAO. Luego, muestre cómo se instanciaría este repositorio dentro de una clase que extienda de `Application` usando el patrón `by lazy`.

2. (3 pts) Lógica Dinámica de Identidad

- a) Cree una data class `Usuario` con los atributos: `nombre (String)`, `apellidos (String)` y `edad (Int?)`.
- b) Cree un objeto con sus datos reales.
- c) Escriba una lógica que calcule el "Puntaje de Identidad": la suma del número de letras de su nombre + el número de letras de sus apellidos.
- d) Escriba el código para imprimir el mensaje:

```

    Usuario: [Nombre Completo], tu puntaje de identidad es: [Resultado]
  
```

- e) Use el operador Elvis (`?:`) para asegurar que el puntaje sea 0 si los datos fuesen nulos. "

13. El operador de Flow que permite combinar múltiples fuentes (como GPS y Sensores) para emitir un estado unificado es combineLatest3().

Instrucción: La comprensión de la pila de software es vital para el desarrollo nativo.

1. Gráfico (1.5 pts)

Dibuje el diagrama de las 5 capas de la arquitectura del Android Open Source Project (AOSP) en el orden jerárquico correcto.

2. Identificación Total (1.5 pts)

Nombre y describa brevemente la función de cada una de las 5 capas que graficó, explicando específicamente qué componentes residen en la capa de Android Runtime (ART).

Se evaluará la precisión terminológica (por ejemplo, no confundir Thread con Coroutine) y el respeto a la jerarquía de capas de la arquitectura Android moderna.

III 1. class UsuarioRepository (

2.

```
a) data class Usuario {  
    string nombre,  
    string apellidos,  
    int edad?  
}
```

```
b) usuario = new usuario (  
    nombre = "Hector",  
    apellidos = "Martinez Huaman",  
    edad = 21  
);
```

```
c) int fun PontajeIdentidad (string nombre, string apellidos) {  
    a1 = Calculo Caracteres (nombre),  
    a2 = Calculo Caracteres (apellidos),  
    return a1 + a2;  
}
```

```
d) printf ("Usuario: %s, nombre: %s, apellido: %s, tu pontaje de identidad es: %d",  
    usuario.nombre, usuario.apellidos, PontajeIdentidad(usuario.nombre, usuario.apellidos));
```

1.

